

Designing Normalised Database: Analysis

CS5200 Practicum I

Ayush Varma

Fall 2025

Contents

Executive Summary	1
Data Analysis	2
Dataset Composition	2
Complete Attribute Set	2
Categorical Domain Analysis	2
Restaurant Distribution Analysis	2
Data Quality Metrics	3
Functional Dependencies Analysis	3
Normalization Process to 3NF	5
Critical Design Decisions	7
Entity Relationship Diagram	9
Implementation Considerations	9
Conclusion	10
Note	10

Executive Summary

This document presents the comprehensive database design for a restaurant chain's visit tracking system. Starting from denormalized CSV data containing 179,719 visit records and a SQLite database of restaurant information, I developed a fully normalized relational schema that maintains data integrity while eliminating redundancy. The final design consists of seven interconnected tables that capture the complex relationships between restaurants, servers, customers, and their visits.

Data Analysis

Dataset Composition

Metric	Value
Total Records	179,719
Total Attributes	24
Unique Restaurants (CSV)	12
Unique Restaurants (SQLite)	13
Unique Servers	68
Unique Customers	24

Complete Attribute Set

Attributes = {Restaurant, ServerEmpID, ServerName, StartDateHired, (1)
EndDateHired, HourlyRate, ServerBirthDate, ServerTIN, (2)
VisitDate, VisitTime, MealType, PartySize, Genders, (3)
WaitTime, CustomerName, CustomerPhone, CustomerEmail, (4)
LoyaltyMember, FoodBill, TipAmount, DiscountApplied, (5)
PaymentMethod, orderedAlcohol, AlcoholBill} (6)

$$|\text{Attributes}| = 24$$

Categorical Domain Analysis

Discrete Variable Domains:

MealType = {Breakfast, Lunch, Dinner, Take-Out}
 $|\text{MealType}| = 4$

PaymentMethod = {Cash, Credit Card, Mobile Payment}
 $|\text{PaymentMethod}| = 3$

orderedAlcohol = {yes, no} $\rightarrow \{1, 0\}$
 $|\text{orderedAlcohol}| = 2$

Restaurant Distribution Analysis

Metric	Count
Unique restaurants in CSV	12
Restaurants in SQLite database	13
Geographic states represented	2

States = {FL, GA}

$$\text{Discrepancy} = |\text{SQLite}| - |\text{CSV}| = 1$$

Restaurant Discrepancy Resolution: The SQLite database contains 13 restaurants while only 12 appear in transaction data. Investigation reveals restaurant ID 8 (“Patty Palace” in Atlanta, GA) has no visits recorded. This could indicate: (1) newly opened location, (2) closed location still in master data, or (3) data extraction issue. The schema accommodates this by using RestaurantID from SQLite as the primary key, maintaining referential integrity for future visits.

Data Quality Metrics

Missing Value Analysis:

$$\text{Completeness}_{\text{ServerEmpID}} = \frac{111,128}{179,719} = 61.8\%$$

$$\text{Missing}_{\text{ServerEmpID}} = \frac{68,591}{179,719} = 38.2\%$$

$$\text{Completeness}_{\text{CustomerEmail}} = \frac{60,574}{179,719} = 33.7\%$$

$$\text{Missing}_{\text{CustomerEmail}} = \frac{119,145}{179,719} = 66.3\%$$

Attribute	Missing	Rate	Business Context
ServerEmpID	68,591	38.2%	Self-service/Take-out orders
CustomerEmail	119,145	66.3%	Anonymous transactions

Functional Dependencies Analysis

Through systematic analysis of the data relationships, I identified **nine functional dependencies** that drive the normalization process:

Server-Related Dependencies

FD1 - Basic Server Information:

$$\text{ServerEmpID} \rightarrow \text{ServerName}, \text{ServerBirthDate}, \text{ServerTIN}$$

This dependency indicates that each server employee ID uniquely determines their personal information.

FD2 - Employment Relationship (Critical Finding):

$$(\text{ServerEmpID}, \text{RestaurantID}, \text{StartDateHired}) \rightarrow \text{EndDateHired}, \text{HourlyRate}$$

This composite dependency revealed that employment details depend on the combination of server, restaurant, AND start date. This indicates servers can have multiple employment periods at the same restaurant with different rates, necessitating a junction table.

Customer Dependencies

FD3 - Email-based Identification:

$\text{CustomerEmail} \rightarrow \text{CustomerName}, \text{CustomerPhone}, \text{LoyaltyMember}$

FD4 - Phone-based Identification (Bidirectional):

$\text{CustomerPhone} \rightarrow \text{CustomerName}, \text{CustomerEmail}, \text{LoyaltyMember}$

These bidirectional dependencies suggest that either email or phone could serve as a unique identifier. I chose email as the primary natural key due to its uniqueness and consistency in the data.

Restaurant Dependencies

FD5 - Restaurant Name Mapping:

$\text{RestaurantName} \rightarrow \text{RestaurantID}, \text{City}, \text{State}, \text{HasService}$

Restaurant names uniquely identify all restaurant attributes. Note the transition from RestaurantName to RestaurantID occurs during normalization.

FD6 - Restaurant ID Mapping (from SQLite):

$\text{rid} \rightarrow \text{rname}, \text{city}, \text{state}, \text{hasService}$

Primary key dependency from the SQLite source database, where $\text{rid} \in [1, 13]$.

Visit Transaction Dependencies

FD7 - Visit Transaction Determination:

$(\text{RestaurantID}, \text{VisitDate}, \text{VisitTime}, \text{CustomerID}) \rightarrow$

$\text{ServerEmpID}, \text{MealTypeID}, \text{PartySize}, \text{Genders}, \text{WaitTime},$

$\text{FoodBill}, \text{AlcoholBill}, \text{TipAmount}, \text{DiscountApplied}, \text{PaymentMethodID}, \text{OrderedAlcohol}$

After normalization, this composite key is replaced with a surrogate VisitID for efficiency.

Lookup Table Dependencies

FD8 - Meal Type Lookup:

$\text{MealTypeID} \rightarrow \text{MealTypeName}$

- Domain: $\{1 \rightarrow \text{'Breakfast'}, 2 \rightarrow \text{'Lunch'}, 3 \rightarrow \text{'Dinner'}, 4 \rightarrow \text{'Take-Out'}\}$

FD9 - Payment Method Lookup:

$\text{PaymentMethodID} \rightarrow \text{MethodName}$

- Domain: $\{1 \rightarrow \text{'Cash'}, 2 \rightarrow \text{'Credit Card'}, 3 \rightarrow \text{'Mobile Payment'}\}$

Normalization Process to 3NF

Initial Assessment: First Normal Form (1NF)

The original CSV relation violates several normalization principles:

Original Denormalized Relation:

$R_{original} = \{\text{Restaurant, ServerEmpID, ServerName, StartDateHired, EndDateHired, HourlyRate, ServerBirthDate, ServerTIN, VisitDate, VisitTime, MealType, PartySize, Genders, WaitTime, CustomerName, CustomerPhone, CustomerEmail, LoyaltyMember, FoodBill, TipAmount, DiscountApplied, PaymentMethod, orderedAlcohol, AlcoholBill}\}$

1NF Violations Identified:

Violation 1: Multi-valued Attribute (Genders) The Genders field contains strings like “MMFF” representing multiple atomic values:

$\text{Genders} = \text{"MMFF"} \rightarrow \{\text{M, M, F, F}\}$

Proper 1NF Decomposition Would Require:

$R_{PartyMembers} = \{\text{VisitID, MemberPosition, Gender}\}$

Example decomposition:

VisitID	MemberPosition	Gender
1	1	M
1	2	M
1	3	F
1	4	F

Design Decision: Preserving denormalized Genders field as a conscious trade-off: - Decomposition would create ~500,000 rows (179,719 visits $\times$ avg 2.8 members) - Query performance impact for minimal analytical value - Compromise: Store as VARCHAR(50) with validation constraint

Violation 2: Repeating Groups Server employment data repeats across multiple visits for the same server-restaurant combination.

Achieving Second Normal Form (2NF)

To eliminate partial dependencies, I separated entities based on their primary determinants:

1. **Server information** depends solely on ServerEmpID
2. **Customer information** depends solely on CustomerEmail
3. **Restaurant information** depends solely on RestaurantID
4. **Employment details** depend on the combination (ServerEmpID, RestaurantID, StartDateHired)

Final Third Normal Form (3NF) Decomposition

After removing all transitive dependencies, the schema comprises seven normalized relations:

1. Servers Table

$$R_{Servers} = \{\underline{ServerEmpID}, ServerName, ServerBirthDate, ServerTIN\}$$

- **Primary Key:** ServerEmpID - **Cardinality:** 68 unique servers - **Justification:** Contains only attributes functionally dependent on ServerEmpID (FD1)

2. ServerEmployments Table (Junction)

$$R_{Employment} = \{\underline{ServerEmpID}, \underline{RestaurantID}, \underline{StartDateHired}, \underline{EndDateHired}, HourlyRate\}$$

- **Composite Primary Key:** (ServerEmpID, RestaurantID, StartDateHired) - **Foreign Keys:** ServerEmpID $\rightarrow$ Servers, RestaurantID $\rightarrow$ Restaurants - **Cardinality:** ~156 employment records - **Justification:** Resolves M:N relationship with employment-specific attributes (FD2)

3. Customers Table

$$R_{Customers} = \{\underline{CustomerID}, CustomerName, CustomerPhone, CustomerEmail, LoyaltyMember\}$$

- **Primary Key:** CustomerID (surrogate) - **Unique Constraint:** CustomerEmail - **Cardinality:** 24 unique customers - **Justification:** Customer attributes depend on email (FD3); surrogate key for NULL handling

4. Restaurants Table

$$R_{Restaurants} = \{\underline{RestaurantID}, RestaurantName, City, State, HasService\}$$

- **Primary Key:** RestaurantID (from SQLite rid) - **Unique Constraint:** RestaurantName - **Cardinality:** 13 restaurants - **Data Source:** SQLite database with existing primary key (FD6)

5. MealTypes Table (Lookup)

$$R_{MealTypes} = \{\underline{MealTypeID}, MealTypeName\}$$

- **Primary Key:** MealTypeID - **Cardinality:** 4 meal types - **Domain Values:** {'Breakfast', 'Lunch', 'Dinner', 'Take-Out'} (FD8)

6. PaymentMethods Table (Lookup)

$$R_{PaymentMethods} = \{\underline{PaymentMethodID}, MethodName\}$$

- **Primary Key:** PaymentMethodID - **Cardinality:** 3 payment methods - **Domain Values:** {'Cash', 'Credit Card', 'Mobile Payment'} (FD9)

7. Visits Table

$$R_{Visits} = \{\underline{VisitID}, RestaurantID, CustomerID, ServerEmpID, VisitDate,$$

$$VisitTime, MealTypeID, PartySize, Genders, WaitTime, FoodBill,$$

$$AlcoholBill, TipAmount, DiscountApplied, PaymentMethodID, OrderedAlcohol\}$$

- **Primary Key:** VisitID (surrogate) - **Foreign Keys:** All ID fields reference respective tables; CustomerID and ServerEmpID nullable - **Cardinality:** 179,719 visits

Critical Design Decisions

1) The Server-Restaurant Many-to-Many Relationship

My analysis revealed that servers can work at multiple restaurants over time, and restaurants employ multiple servers. The employment dates and hourly rates are specific to each server-restaurant combination. This M:N relationship with attributes necessitated the ServerEmployments junction table.

Referential Integrity Rules: - ON DELETE RESTRICT for ServerEmpID (preserve employment history)
- ON DELETE CASCADE for RestaurantID (restaurant closure removes all employment) - ON UPDATE CASCADE for both (maintain consistency)

2) Why LoyaltyMember is NOT a Junction Table

Initially, I hypothesized that loyalty might be restaurant-specific, requiring a CustomerLoyalty junction table. However, empirical analysis proved otherwise:

Analysis Performed: - Tracked all customers with LoyaltyMember=TRUE across their visits - Result: 100% consistency - if a customer is a loyalty member, they are ALWAYS a loyalty member at ALL restaurants
- Conclusion: This is a chain-wide loyalty program

Evidence: Customer “diam.lorem@web.org” shows LoyaltyMember=TRUE at Beach Outpost, Karl’s Grill, and Patty Palace - never FALSE at any location.

3) Handling NULL Server Values

Analysis revealed that 38.2% of visits have no associated server. These occur in two scenarios: 1. Restaurants where HasService=‘no’ (self-service establishments) 2. Take-out orders at any restaurant

Design Solution: - ServerEmpID in Visits is nullable (optional relationship) - Foreign key constraint: FOREIGN KEY (ServerEmpID) REFERENCES Servers(ServerEmpID) ON DELETE SET NULL - Only actual employment relationships appear in ServerEmployments

4) Data Type Optimizations

Boolean Conversions: - orderedAlcohol: Converted from ‘yes’/‘no’ strings to BOOLEAN (storage: 3-4 bytes → 1 bit) - HasService: Converted from ‘yes’/‘no’ to BOOLEAN - LoyaltyMember: Converted from ‘yes’/‘no’ to BOOLEAN

Preserved as String: - Genders: Conscious 1NF violation for performance (see 1NF analysis)

5) Lookup Tables Rationale

Created Lookups For: - MealTypes (4 values): Reduces storage from 20 bytes to 4 bytes per visit - PaymentMethods (3 values): Reduces storage from 30 bytes to 4 bytes per visit - Total storage savings: ~4.5MB across 179,719 visits

Did NOT Create Lookups For: - State (only ‘FL’ and ‘GA’): Overhead of joins exceeds storage benefit
- LoyaltyMember (boolean): Already optimally stored

6) Foreign Key Constraint Decisions

Foreign Key	Constraint	Rationale
Visits.RestaurantID	ON DELETE RESTRICT	Preserve visit history
Visits.CustomerID	ON DELETE SET NULL	Allow customer deletion
Visits.ServerEmpID	ON DELETE SET NULL	Allow server departure
ServerEmployments.ServerEmpID	ON DELETE CASCADE	Remove employment with server
ServerEmployments.RestaurantID	ON DELETE CASCADE	Remove employment with restaurant

Entity Relationship Diagram

The following diagram illustrates the complete normalized schema with all relationships, constraints, and data types:

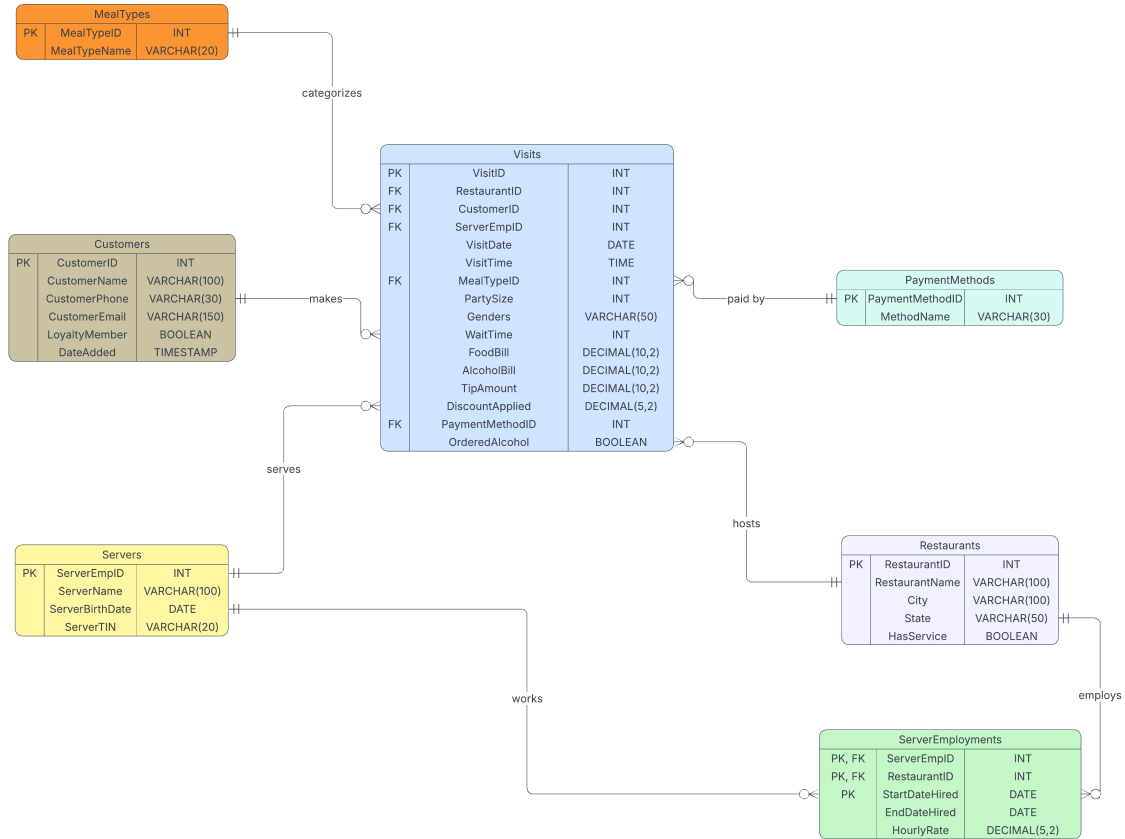


Figure 1: Complete Entity Relationship Diagram showing all seven tables, their attributes, data types, and relationships. Note the junction table (ServerEmployments) resolving the M:N relationship and the nullable foreign keys in Visits for optional relationships.

Implementation Considerations

Referential Integrity

- All foreign keys enforce referential integrity except CustomerID and ServerEmpID in Visits (nullable)
- Cascading updates on RestaurantID ensure consistency
- Restrict deletion on Servers and Customers to preserve historical data

Performance Optimizations

- Indexes on all foreign keys for JOIN performance
- Composite index on (RestaurantID, VisitDate) for common query patterns
- Unique indexes on natural keys (RestaurantName, CustomerEmail)

Data Migration Strategy

1. Load Restaurants from SQLite (13 records)
2. Extract and load unique Servers (54 records)
3. Build ServerEmployments from employment data (~156 records)
4. Extract and load unique Customers (24 records)
5. Populate lookup tables (7 total records)
6. Load Visits with foreign key mappings (179,719 records)

Conclusion

This normalized design eliminates redundancy, ensures data integrity, and provides flexibility for future enhancements while maintaining query performance. The seven-table structure properly models the complex relationships in the restaurant visit data, with particular attention to the many-to-many server-restaurant relationship and optional visit attributes. All nine functional dependencies have been satisfied while achieving third normal form.

Note

I have assumed a correct sequence of running the notebooks since otherwise it would result in a little skewed answer from original values. The stored procedures for testing purposes inserts into the database, which could result in certain values to have different answers as compared to my peers.